

Partially Registered Multi-authority Attribute-based Encryption

Viktória I. Villányi¹, Vladimir Božović²

¹ Department of Computer Algebra, Eötvös Loránd University, Budapest, Hungary
vvillanyi@inf.elte.hu

² University of Montenegro, 2 Cetinjski Put, Podgorica 81000, Montenegro
vladimirb@ucg.ac.me

Abstract. Attribute-based encryption can be considered a generalization of public key encryption, enabling fine-grained access control over encrypted data using predetermined access policies. In general, we distinguish between key-policy and ciphertext-policy attribute-based encryption schemes. Our new scheme is built upon the multi-authority attribute-based encryption with an honest-but-curious central authority scheme in a key-policy setting presented earlier by Božović et al., and it can be considered an extension of their scheme. In their paper, trust was shared between the central authority and the participating authorities, who were responsible for issuing attribute-specific secret keys. The central authority was not capable of decrypting any message as long as there exists an honest attribute authority. In our new scheme, we maintain this feature, and add another level of security by allowing users to participate in the key generation process and contribute to the final user-specific attribute secret keys. Users gain more control over their own secret keys, and they will be the only parties with access to the final user-specific secret keys. Furthermore, no secure channels, only authenticated communication channels are needed between users and authorities. After the modifications our scheme will be closer to the registered multi-authority attribute-based encryption. We refer to our scheme as a partially registered multi-authority attribute-based encryption scheme. We prove the security of our scheme in the Selective-ID model.

1 Introduction

The family of identity-based, attribute-based, and functional encryption schemes can be traced back to 1985 when Shamir proposed the original concept of identity-based encryption schemes [Sha85]. Boneh and Franklin [BF03] presented the first identity-based encryption scheme that uses bilinear pairing as a basic building block. They also defined a security model for identity-based encryption and proved that the constructed scheme whose security was based on the Bilinear Diffie-Hellman problem was secure. Canetti et al. [CHK03] proposed a slightly weaker security model, called selective identity model. In their model the adversary must commit to the identity it intends to attack before the public parameters are generated. An attribute-based encryption scheme capable of handling

multiple authorities was first proposed by Chase in 2007 [Cha07]. Chase’s scheme built upon the earlier single-authority attribute-based encryption scheme from Sahai and Waters [SW05]. Sahai and Waters showed that their Fuzzy Identity-based Encryption can be used for a new type of application that they named for “attribute-based encryption”. Attribute-based encryption can be considered a generalization of public key encryption, enabling fine-grained access control over encrypted data using predetermined access policies. We can distinguish between key-policy attribute-based encryption schemes and ciphertext-policy attribute-based encryption schemes. In key-policy attribute-based encryption schemes, attributes describe the encrypted data, and policies are embedded in the user’s keys. In this case, a secret (related to a defined access structure) is shared by the authorities, and multiple parties could be involved in the secret sharing process. Conversely, in ciphertext-policy attribute-based encryption schemes, attributes describe the user’s credentials, and the encryptor specifies a policy to determine who can decrypt the data. In this latter case, secret sharing (related to a defined access structure) is done by the encryptor, and there is no need to involve other parties.

Our new scheme is based on the multi-authority scheme in the key-policy settings by Božović et al. [BSSV12], which enhanced the security of Chase’s scheme [Cha07]. In our scheme, as in the earlier multi-authority schemes, the trust is distributed among multiple authorities. However, we succeeded in reducing the trust placed in authorities and giving users more control over their own set of attribute secret keys. Our new protocol still requires key generating authorities instead of a key curator, who does not hold a secret. While our scheme is not fully registered attribute-based encryption scheme, it is a step closer to these type of encryption schemes. We refer to our scheme as a partially registered multi-authority attribute-based encryption scheme. The authorities will have some partial information about the attribute secret keys, but they will not have access to the complete user specific decryption keys. We prove the security of our scheme in the Selective-ID model, relying on the hardness of the Decisional Bilinear Diffie-Hellman assumption.

We summarize here how our new scheme enhances its predecessor [BSSV12]:

- Users are able to contribute to the randomness of their own attribute secret keys. Only the users know their final updated attribute secret keys.
- The central authority interacts with the users instead of the attribute authorities, and it issues attribute secret keys to users similar to the other authorities, and it publishes one extra attribute value for encryption, instead of the publishing dummy public keys for every single user as in the previous protocol.
- The central authority shares some extra information with the users, enabling them to check the proper updates of their secret keys and eliminate cheating authorities from the protocol.
- Between the users and the authorities no secure channels, only an authenticated channels are needed. A secure channels are needed only between the central authority and the attribute authorities.

2 Related literature

In attribute-based encryption schemes, the primary role of attribute authorities is to generate user-specific attribute secret keys. They can keep those keys for the lifetime of the system. They can be the target of different attacks and provide points of failure of the systems. There is a broad literature on models and attempts to decrease trust and the role of the authorities. Melissa Chase [Cha07] designed the very first multi-authority attribute-based encryption scheme that was able to combine private values from multiple authorities and issue secret keys for users. She introduced a so-called central authority (CA). The CA has access to the secret keys of all users and the system’s master secret key, which enables the CA to decrypt any messages, regardless of any later defined access structure. The CA issues helper secret keys to the users for decryption. Later, Chase et al. [CC09], and Lewko et al. [LW11] improved the original scheme. In the improved scheme [LW11], the authors succeeded in removing the CA and the helper secret key generation. Božović et al. [BSSV12] removed some partial trust from the central authority (honest-but-curious), and they succeeded in sharing the trust between the authorities (multi-authority system).

Their new assumption was that the central authority follows the protocol but he tries to gather information about the encrypted messages. In the presented protocol, the central authority is no longer capable of decrypting any messages.

The notion of registration-based encryption (RBE) was introduced by Garg et al. [GHMR18] with the goal of removing the private-key generator (PKG) from identity-based encryption. They introduced a new type of participant in the protocol, the “key curator” (KC) instead of the private-key generator (which was a single point of failure). In an RBE scheme, users sample their own public and secret keys. The KC aggregates the public keys of all registered users into a public key for the system and updates the public parameter whenever a new user joins the system. The recipient’s identity and the aggregated public key can be used to encrypt the messages. The decryption process may require additional information (similar to the helper secret keys) from the KC.

A problem with malicious key curator in registration-based encryption schemes was addressed by Goyal et al. [GV20]. In their paper, they initiated a formal study of registration based encryption systems with a malicious key curator. They introduced the so-called verifiable registration-based encryption schemes. Their verifiable registration-based encryption system additionally gives the users the capability to obtain short proofs from the key accumulator, proving correct (and unique) registration for every registered user as well as proving non-registration for any yet unregistered identity.

Malicious key curator could cause problems in attribute-based encryption schemes as well. Solving the malicious key curator problem is a more complex problem in the case of attribute-based encryption since identities are defined by the set of attributes, and collusion between such identities should be avoided.

In 2023 Hohenberger et. al. [HLWW23] proposed a new concept in context of attribute-based encryption, the registered attribute-based encryption. They generalized RBE and developed the registered attribute-based encryption (RABE)

to address the key escrow problem and remove the need for long-term secret keys. In their proposed scheme, they reintroduced the helper secret key. The helper secret keys provide additional information that allows users to decrypt messages. The helper secret key connects the user’s decryption keys and the common public key and it is issued by the key curator. To encrypt a message, a common public key is needed, which depends on the users’ secret keys. Users generate their own secret and public key pairs based on a structured Common Reference String. With the help of the key curator, they combine these values to generate the common public key for encryption. The key curator combines public keys of all users and issues helper secret keys. The public key needs frequent updates when a new user joins the system. A natural solution is to update the public key and the helper secret keys every time a new user joins. They provided a more efficient solution in their paper. The basic idea is that there exist several public keys in the system, and when a new user joins, they update only one of the many public keys. The weakness is that every message has to be encrypted with every single public key. With this idea they avoided the need for updating every single helper secret key after a new user joins, but it increases the length of the ciphertext, affecting the efficiency of the scheme. Another limitation of their scheme is that it requires a (structured) Common Reference String whose size scales quadratically with the number of users and multiplicatively with the size of the attribute universe. This constraint limits the size of the users and attribute spaces. In a later paper [GLWW24], Rachit Garg et al. tried to fix this weakness of the scheme. Numerous papers on registered attribute-based encryption [ZZGQ23, AT24] and registration-based encryption [FKdP23, GKMR23, CHH⁺24] have been published in the past year. This research topic remains highly active and continues to generate significant research interest. We developed a different solution to avoid frequent update of the keys. In our scheme, we have a common public key for all users that does not contain the individual secret keys of the users. It is calculated from the public keys of the attribute authorities and the central authority. The secret keys related values of the individual users are removed one step earlier during the decryption (by using attribute secret keys and helper secret keys). The individual secret values are not tied to the encryption exponent but to the attribute secret keys and to the $X_{k,u}$ values. The $X_{k,u}$ values play the role of helper secret keys in our scheme. The $X_{k,u}$ helper value is computed from the input of the central authority and the attribute authority, and it is updated by the user. These three entities jointly generate the secret values. As long as one attribute authority is honest, the scheme is proven to be secure even if the central authority is honest-but-curious. When a new user joins the our system, there is no need to update the public or secret key pairs of existing users, nor the authority related keys. Since our construction does not rely on a Common Reference String (CRS), there are no predefined limitations on the size of the attribute set or the number of users, as these are typically tied to the size of the CRS.

2.1 Registered attribute-based encryption scheme with malicious key curator

As previously mentioned, the presence of a malicious key curator in registered attribute-based encryption poses security concerns. The authors of the [HLWW23] paper suggested a generic solution to solve the problem. They introduced a potential auditor into the system when the key curator is malicious. If the key curator deviates from the protocol, an auditor may be employed to detect such malicious behavior. This, however, raises several questions concerning the auditing process:

- Can audits be performed in parallel to speed up the process?
- How long should auditing continue (e.g. life-long)?
- Should every user audit the key curator?
- Can we trust a single auditor (instead of a key curator)?
- Do we need multiple auditors to ensure reliability?
- Is the system secure as long as at least one auditor remains honest?

These questions started to resemble earlier ones we had asked about attribute-based encryption schemes in the multi-authority setting. The security of our new partially registered multi-authority attribute-based scheme is proved under a similar assumption: the scheme remains secure as long as at least one authority remains honest.

In the ePrint version of the paper [HLWW23], the authors noted that it is impossible to prevent a malicious key curator from registering a set of attributes that would enable them to decrypt any ciphertext. They claimed, that such behavior is always detectable by an external auditor. In Appendix A, we explain how a malicious key curator could construct a scheme that allows them to decrypt any message. In the rest of the paper, we present our partially registered multi-authority attribute-based encryption scheme, whose security relies on multiple authorities and distributes trust between them.

3 Notation and preliminaries

The security of our new scheme based on the Decisional Bilinear Diffie-Hellman assumption. This section covers the necessary terminology related to bilinear maps and partially registered multi-authority attribute-based encryption. The security model is discussed in Section 3.3, where, similar to [BSSV12], we employ the selective-ID model.

3.1 Bilinear maps and the Bilinear Diffie-Hellman assumption

Let G_1 and G_2 be groups of prime order p , with P as a generator of G_1 . The prime p is assumed to be superpolynomial in the security parameter ℓ , and all operations in G_1 and G_2 are assumed to be computable efficiently, i.e., in probabilistic polynomial time. We adopt additive notation for G_1 and multiplicative

notation for G_2 . An admissible bilinear map is denoted by $e : G_1 \times G_1 \rightarrow G_2$, which satisfies the following properties [BF03]:

- For all $P, Q \in G_1$ and for all $\alpha, \beta \in \mathbb{Z}$, it holds that $e(\alpha P, \beta Q) = e(P, Q)^{\alpha\beta}$.
- The map $e(P, P) \neq 1$, meaning $e(P, P)$ is a generator of G_2 .
- There is a probabilistic polynomial time algorithm to compute $e(P, Q)$ for any $P, Q \in G_1$.

In this framework, the Decisional Bilinear Diffie-Hellman (D-BDH) problem for (G_1, G_2, e) is the problem of distinguishing the output of the following experiment: The challenger selects $\alpha, \beta, \gamma, \eta \leftarrow \{0, 1, \dots, p-1\}$ independently and uniformly at random, flips a fair coin $\delta \leftarrow \{0, 1\}$, and then outputs the tuple

$$(P, \alpha P, \beta P, \gamma P, e(P, P)^{\delta \cdot \alpha \beta \gamma + (1-\delta) \cdot \eta}).$$

In this experiment, with probability $1/2$ the last component of the challenger's output is $e(P, P)^{\alpha\beta\gamma}$, and with probability $1/2$ the last component is a uniformly at random chosen element from G_2 . The *advantage* of algorithm $\mathcal{A}$ in solving the D-BDH problem is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{bdh}}(\ell) := \Pr(\delta' = \delta) - \frac{1}{2},$$

where δ' is the output of $\mathcal{A}$ when trying to guess the value of the fair coin δ . An algorithm $\mathcal{A}$ has a *non-negligible advantage* in solving the D-BDH problem if $\text{Adv}_{\mathcal{A}}^{\text{bdh}}$ is not negligible³ where the probability is over the randomly chosen $\alpha, \beta, \gamma, \eta$ and the random bits used by $\mathcal{A}$.

Definition 1 (Decisional Bilinear Diffie-Hellman assumption). *The Decisional Bilinear Diffie-Hellman assumption holds for (G_1, G_2, e) if there exists no probabilistic polynomial time algorithm having non-negligible advantage in solving the above D-BDH problem.*

3.2 Authorities, attributes and users

Let $\mathcal{K}$ be the set of attribute authorities of polynomial size, and let $\mathcal{U}$ be set of users of polynomial size. We denote by $\mathcal{A}_k$ the polynomial size set of attributes handled by attribute authority $k \in \mathcal{K}$ and $\mathcal{A}_{k_{\text{CA}}}$ is a set of a single attribute managed by the central authority. It is required that the sets $\mathcal{A}_k$ and $\mathcal{A}_{k_{\text{CA}}}$ are pairwise disjoint. Therefore, the *universal attribute set*

$$\mathcal{A} := \bigsqcup_{k \in \mathcal{K}} \mathcal{A}_k \cup \mathcal{A}_{k_{\text{CA}}}$$

is the disjoint union of the attribute sets. The central authority $k_{\text{CA}} \notin \mathcal{K}$ which we will model as honest-but-curious central authority k_{CA} honestly follows the

³ We refer to a function $f : \mathbb{N}_{>0} \rightarrow \mathbb{R}$ as negligible, if $|f| = |f(\ell)| \in \frac{1}{\ell^{o(1)}}$.

protocol, but will try to decrypt ciphertexts sent by users in the system. Communication is allowed during an initialization phase between k_{CA} and k for each authority $k \in \mathcal{K}$ and for each user but we do not allow any communication among the authorities in $\mathcal{K}$.

To distinguish different users, we assume that each user $u \in \mathcal{U}$ has a unique identifier. The set of attributes in $\mathcal{A}$ available to user $u \in \mathcal{U}$ is denoted by $\mathcal{A}_u$. Similarly, the set of attributes associated with a ciphertext C denoted by $\mathcal{A}_C$. This set $\mathcal{A}_C$ is chosen by the encrypting party as part of the input to the encryption algorithm, the other part of the input being the plaintext. Each authority $k \in \mathcal{K}$ has an associated threshold $d_k \in \mathbb{N}_{>0}$. The users u whose attribute sets satisfy

$$|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| \geq d_k \text{ for every } k \in \mathcal{K}$$

and obtains $\mathcal{A}_{k_{CA}}$ from the k_{CA} are able to decrypt the ciphertext C . This means that, for each authority k , user u must possess at least d_k of the attributes that have been specified at the time of encryption. To decrypt a ciphertext, user $u \in \mathcal{U}$ uses the secret keys jointly generated with the authorities. Now, we give the formal definition of the partially registered multi-authority attribute-based encryption.

Definition 2 (Partially Registered Multi-authority Attribute-based Encryption (PR-MABE)).

Setup A probabilistic polynomial time algorithm that, given the security parameter 1^ℓ , a list of pairwise disjoint sets of attributes $[\mathcal{A}_k]_{k \in \mathcal{K}} \cup \mathcal{A}_{k_{CA}}$ and thresholds $[d_k]_{k \in \mathcal{K}}$ generates a public key and secret key pair for each authority and public system parameters.

Central Authority's User Specific Key Generation (CKGen) A probabilistic polynomial time algorithm that, given the number of attribute authorities, a unique identifier of a user u , and the central authority's secret key, outputs a user and the authority specific shares of the central authority's secret key, as well as the user specific secret key for the attribute $\mathcal{A}_{k_{CA}}$.

User Attribute Key Generation (UKGen) A probabilistic polynomial time algorithm that, given an attribute authority k 's threshold d_k , a unique identifier of a user u , a set $\mathcal{A}_u \cap \mathcal{A}_k$ outputs the user generated part of the attribute secret keys related to $\mathcal{A}_k$.

Attribute Authority Key Generation (AKGen) A probabilistic polynomial time algorithm that, given an attribute authority k 's threshold d_k , a unique identifier of a user u , a set $\mathcal{A}_u \cap \mathcal{A}_k$ outputs the $\mathcal{A}_k$ generated part of the attribute secret keys.

User Attribute Key Update 1 (Update1) A probabilistic polynomial time algorithm that, given a unique identifier of user u , along with the outputs of the UKGen, AKGen, and CKGen algorithms (where CKGen provides the

user and authority specific shares of the central authority's secret key), outputs the updated, masked secret keys for user u .

User Attribute Key Update 2 (Update2) A probabilistic polynomial time algorithm that, given a unique identifier of user u , along with the outputs of the UKGen (provides user u 's secret values) and Update1 algorithms, outputs the final updated secret keys for user u .

Encryption (Enc) A probabilistic polynomial time algorithm that, given the master public key mpk , attributes $\mathcal{A}_{k_{CA}}, \mathcal{A}_C \subset \biguplus_{k \in \mathcal{K}} \mathcal{A}_k$ and a message, outputs the ciphertext C .

Decryption (Dec) A deterministic polynomial time algorithm that, given a set of secret keys for a set of attributes $\mathcal{A}_{k_{CA}} \cup \mathcal{A}_u$, a ciphertext C encrypted with attribute set $\mathcal{A}_C$, outputs the corresponding plaintext M if $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| \geq d_k$ for all attribute authorities $k \in \mathcal{K}$; otherwise it outputs an error symbol $\perp$.

Correctness Partially registered multi-authority attribute-based encryption scheme is said to be correct if, whenever a ciphertext is proper encryption of a message M with the master public key mpk under the defined attribute set, and a set of secret keys is obtained from the key generation algorithm and updated by the users for a set of attributes satisfying the given access structure, the decryption of the ciphertext equals the original message.

Remark 1. The attribute secret keys for users are generated by the attribute authorities $k \in \mathcal{K}$ and by the users together. In the security model we will not give the adversary the possibility to query k_{CA} for secret user keys.

A crucial feature of a multi-authority attribute-based encryption scheme is the prevention of collusions among users: we want to prevent any set of users, each of which is not able to decrypt a ciphertext C , from combining their information to decrypt C .

3.3 Security model

Like [BSSV12], we use a selective-ID model for the security analysis. The adversary $\mathcal{H}$ has to specify the set of attributes that he wants to attack before receiving any public keys of the system. As in [BSSV12], for our security analysis we impose the technical restriction that the adversary does not query the same attribute authority twice for secret keys of the same user. (The adversary cannot issue corrupt honest key query and register corrupted key query with the same attribute authority for secret keys of the same user.) We define the security of our partially registered multi-authority attribute-based (PR-MABE) of encryption. An adversary has to win in the following game to defeat the security of our scheme.

Definition 3 (Security of PR-MABE in the selective-ID model).

Let $\Pi_{RABE} = (\text{Setup}, \text{CKGen}, \text{UKGen}, \text{AKGen}, \text{Update1}, \text{Update2}, \text{Enc}, \text{Dec})$ be a partially registered attribute-based encryption in the selective-ID model with an attribute set $\mathcal{A}$, an access structure and a message space $\mathcal{M}$. For security parameter 1^ℓ and adversary $\mathcal{H}$ and a bit $\delta \leftarrow \{0, 1\}$ we define the following game between the challenger and the adversary:

Setup

1. The adversary $\mathcal{H}$ outputs
 - a non-empty list $\mathcal{U}$ of (unique identifiers of) users
 - a non-empty list $\mathcal{K}$ of (unique identifiers of) attribute authorities
 - a list $[(\mathcal{A}_k, \text{corrupted}, d_k)]_{k \in \mathcal{K}}$ of non-empty, pairwise disjoint attribute sets, each along with a threshold $d_k \in \mathbb{N}_{>0}$ and a flag indicating if the respective authority is corrupted. There must be at least one uncorrupted authority.⁴
 - a non-empty set of attributes $\mathcal{A}_C \subseteq \biguplus_{k \in \mathcal{K}} \mathcal{A}_k$ and $\mathcal{A}_{k_{CA}}$ that will be associated with the challenge ciphertext.
2. The public and secret keys are generated, and $\mathcal{H}$ learns
 - the public system parameters
 - the complete history of all those authorities $k \in \mathcal{K}$ that are corrupted.

Register honest key query The adversary $\mathcal{H}$ is allowed to ask the authorities $k \in \mathcal{K}$ to generate honest attribute keys. The challenger generates the attribute secret keys by running the CKGen, UKGen, AKGen, Update1 and Update2 algorithms, or it may simplify the procedure by running only the CKGen and AKGen algorithms (as using the other algorithms does not provide an additional level of security in this case).

Corrupt honest key query The adversary $\mathcal{H}$ can query the authorities $k \in \mathcal{K}$ for honestly generated secret user keys for attributes in $\mathcal{A}_k$ for user u . Whenever the adversary $\mathcal{H}$ queries k for a secret key for attribute $a \in \mathcal{A}_k$ for user u , the attribute a is added to the (initially empty) set $\mathcal{A}_u$. The only restrictions on secret key queries are the following:

- at any time, for each user u , there is at least one uncorrupted authority $\hat{k} = \hat{k}(u)$ with $|\mathcal{A}_u \cap \mathcal{A}_{\hat{k}} \cap \mathcal{A}_C| < d_{\hat{k}}$ ⁵
- for each user u , no authority $k \in \mathcal{K}$ is queried more than once for secret keys of u .

Register corrupted key query The adversary can issue register corrupted key queries for attributes in $\mathcal{A}_k$ for user u . The adversary runs UKGen, Update2 and the challenger runs CKGen, AKGen, Update1 to jointly generate attribute secret keys. Whenever the adversary $\mathcal{H}$ queries k for a register corrupted key for attribute $a \in \mathcal{A}_k$ for user u , the attribute a is added to the (initially empty) set $\mathcal{A}_u$. The restrictions on register corrupted key queries are the same as in the previous case.

Challenge

⁴ Note that the central authority k_{CA} is not included in this list and in particular cannot be corrupted.

⁵ The uncorrupted authority $\hat{k} = \hat{k}(u)$ may be different for each user u .

1. The adversary $\mathcal{H}$ outputs two equal length messages M_0, M_1 .
2. The challenger flips a fair binary coin $\delta \leftarrow \{0, 1\}$ and then applies the encryption algorithm to M_δ and the attribute set $\mathcal{A}_C \cup \mathcal{A}_{k_{CA}}$.
3. The resulting ciphertext C is given to the adversary $\mathcal{H}$.

Further secret key queries The adversary can issue further secret key queries, subject to the same restrictions as before.

Guess The adversary $\mathcal{H}$ outputs a guess δ' for the challenger's secret coin δ .

For a partially registered multi-authority attribute-based encryption scheme to be secure, we require that there is no efficient algorithm achieving a non-negligible advantage in the security game. More specifically, we define the advantage of an adversary $\mathcal{H}$ as

$$\text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell) := \Pr(\delta' = \delta) - \frac{1}{2}.$$

A scheme for partially registered multi-authority attribute-based encryption is secure in the selective-ID model, if for all probabilistic polynomial time adversaries $\mathcal{H}$, the advantage $\text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell)$ is negligible.

The security requirement in this definition does not address the question of which information is available to the central authority. To express a requirement that limits the possibilities of an honest-but-curious central authority, we take a more detailed look at the setup phase, which is combined into a single algorithm in Definition 4.

Remark 2. From a practical perspective, it is desirable to have no communication between attribute authorities, and only very limited interaction of the central authority with each attribute authority. In our new protocol, the central authority communicates with the users when they join the system. The central authority sends one message to each user when they start registering their attributes. The public system parameters will be derived from the parameters that are published by the authorities.

Security against insider attack. We consider an attack from the central authority. We assume the central authority is honest and follows the protocol. From the viewpoint of the central authority, the protocol is very similar to the Božović et al. [BSSV12] protocol. The information available to the central authority is not affected by the modified interactive attribute key generation process. We rely on their security model and proof here. We present the security model here, and we refer to the security proof in their paper. The game in Definition 4 captures a setting where an honest-but-curious central authority tries to violate the indistinguishability of ciphertexts. We introduce a “curious” algorithm $\mathcal{B}$ which, similarly to the “outside adversary” $\mathcal{H}$ in Definition 3, fixes the attribute sets and their distribution among the attribute authorities. Furthermore, $\mathcal{B}$ specifies the set of attributes that will be associated with the challenge ciphertext. At the end of the setup phase, $\mathcal{B}$ learns the complete state of the central authority, and based on this knowledge then tries to violate the indistinguishability of ciphertexts.

Definition 4 (Security of PR-MABE in the selective-ID model when tolerating an honest-but-curious central authority). Let $\Pi_{RABE} = (\text{Setup}, \text{CKGen}, \text{UKGen}, \text{AKGen}, \text{Update1}, \text{Update2}, \text{Enc}, \text{Dec})$ be a partially registered attribute-based encryption in the selective-ID model with an attribute set $\mathcal{A}$, an access structure and a message space $\mathcal{M}$. For security parameter 1^ℓ and adversary $\mathcal{B}$ and a bit $\delta \leftarrow \{0, 1\}$ we define the following game between the challenger and the honest-but-curious adversary:

- Setup**
1. Given the security parameter 1^ℓ , the algorithm $\mathcal{B}$ outputs:
 - a non-empty list $\mathcal{U}$ of (unique identifiers of) users,
 - a non-empty list $\mathcal{K}$ of (unique identifiers of) attribute authorities,
 - a list $[(\mathcal{A}_k, \text{corrupted}, d_k)]_{k \in \mathcal{K}}$ of non-empty, pairwise disjoint attribute sets, each along with a threshold $d_k \in \mathbb{N}_{>0}$ and a flag indicating if the respective authority is corrupted. There must be at least one uncorrupted authority.⁶
 2. The public and secret keys of all authorities $k \in \mathcal{K}$ are generated, and $\mathcal{B}$ learns:
 - all public keys,
 - the public system parameters,
 - the complete history of all those authorities $k \in \mathcal{K}$ that are corrupted,
 - the complete history of the central authority k_{CA} .

Challenge

1. The algorithm $\mathcal{B}$ outputs two equal length messages M_0, M_1 and a non-empty set of attributes $\mathcal{A}_C \subseteq \biguplus_{k \in \mathcal{K}} \mathcal{A}_k$.
2. The challenger flips a fair coin $\delta \leftarrow \{0, 1\}$ and then applies the encryption algorithm to M_δ and the attribute set $\mathcal{A}_C \cup \mathcal{A}_{k_{\text{CA}}}$.
3. The resulting ciphertext C is given to $\mathcal{B}$.

Guess

The algorithm $\mathcal{B}$ outputs a guess δ' for the challenger's secret coin δ .

For an algorithm $\mathcal{B}$, we define the advantage in the game as

$$\text{Adv}_{\mathcal{B}}^{\text{ca}}(\ell) := \Pr(\delta' = \delta) - \frac{1}{2}.$$

A scheme for partially registered multi-authority attribute-based encryption can tolerate an honest-but-curious central authority, if for all probabilistic time algorithms $\mathcal{B}$, the advantage $\text{Adv}_{\mathcal{B}}^{\text{ca}}(\ell)$ is negligible.

Remark 3. Unlike for the adversary $\mathcal{H}$ in Definition 3, we do not require that an honest-but-curious central authority specifies the challenge attributes $\mathcal{A}_C$ in advance: algorithm $\mathcal{B}$ in Tolerating an honest-but-curious central authority definition does not have to provide this set before the challenge phase. The reason for that is that the central authority is not permitted to issue secret key queries.

⁶ Note that the central authority k_{CA} is not included in this list and in particular cannot be corrupted.

4 Partially registered multi-authority attribute-based encryption

Our new protocol builds on the Božović et al. [BSSV12] protocol. As mentioned earlier, the main difference is that the users can actively participate in the attribute secret key generation in our new protocol. Users contribute to the randomness of their attribute secret keys, and only they possess the final updated attribute secret keys and are able to decrypt messages. The key generation process includes the user, who generate similar secret sharing polynomials as the attribute authorities did before. The sum of the two generated polynomials (one by the user, another one by the attribute authority) defines the updated attribute secret keys. Users can decrypt any messages with their updated secret keys if their attributes satisfy the given access policy. The authorities and the central authority cannot decrypt a message as long as there is at least one honest authority in the system. We also introduce another modification to the protocol. In [BSSV12] protocol, the central authority generated user specific elements of its secret key that were sent to the authorities. One more value for every user was published as part of the public system parameters and was referred as a "dummy secret". In this new protocol none of these additional dummy secrets will be published. The user specific value will be shared with the corresponding user. With this change, the new protocol enhances the security of the previous one. An additional feature of our scheme is that the final secret keys are computed and adjusted by their respective owners. These keys never leave the users' premises, thereby ensuring a higher level of security.

4.1 The proposed protocol

Setup.

- The central authority k_{CA} chooses $\alpha, \beta \in \{0, \dots, p-1\}$ uniformly at random and computes $\sigma = \alpha \cdot \beta \mod p-1$, $A = \alpha P \in G$ and $B = \beta P \in G$ values. The central authority chooses a secret value $t_{k_{CA}} \leftarrow \{1, \dots, p-1\}$ uniformly at random and publishes the public system parameter

$$\left(e(P, P)^\sigma, \underbrace{t_{k_{CA}} \cdot P}_{=: T_{k_{CA}}} \right).$$

- Attribute authority $k \in \mathcal{K}$ chooses a value $r_k \leftarrow \{0, \dots, p-1\}$ uniformly at random. Moreover, for each of its attributes $a \in \mathcal{A}_k$, a secret value $t_{k,a} \leftarrow \{1, \dots, p-1\}$ is chosen uniformly at random by k , and the pair

$$\left(e(P, P)^{r_k}, \underbrace{[t_{k,a} \cdot P]_{a \in \mathcal{A}_k}}_{=: T_{k,a}} \right)$$

forms k 's public key. The secret key of attribute authority k contains the aforementioned values r_k and $[t_{k,a}]_{a \in \mathcal{A}_k}$.

Remark 4. The value $e(P, P)^{r_k}$ is only used during encryption to compute the product $\text{pk} \cdot \prod_{k \in \mathcal{K}} e(P, P)^{r_k}$ —which is ciphertext-independent. If one allows the attribute authorities to contribute to the generation of the public system parameters, the $e(P, P)^{r_k}$ -component in the attribute authorities' public keys can be omitted. To do so, the public system parameter $\text{pk} = e(P, P)^\sigma$ can be replaced with $e(P, P)^{\sigma + \sum_{k \in \mathcal{K}} r_k}$.

Attribute key generation To register the secret key associated with an attribute $a \in \mathcal{A}_k \cap \mathcal{A}_u$ for a user $u \in \mathcal{U}$, central authority and attribute authority k proceeds as follows:

- The user u presents their credentials to the central authority. The central authority runs *CKGen* and issues a certificate for the approved credentials. The k_{CA} chooses for each pair $(k, u) \in \mathcal{K} \times \mathcal{U}$, uniformly at random a secret value $s_{k,u} \leftarrow \{0, \dots, p-1\}$ and for each $u \in \mathcal{U}$ computes a user specific secret $s_{k_{\text{CA}},u} := \sigma - \sum_{k \in \mathcal{K}} s_{k,u}$. The central authority sends all encrypted (under the corresponding authority k 's public key) $[s_{k,u} \cdot P]_{k \in \mathcal{K}}$ values to the user u for all involved authorities as well as the central authority specific attribute secret key $D_{k,u} = \frac{s_{k_{\text{CA}},u} \cdot P}{t_{k_{\text{CA}}}}$. The k_{CA} does not store these user specific values. The central authority shares A and B values with the users that are needed for security purpose. Further on, if the central authority shares the $e(P, P)^{s_{k,u}}$ values with the users then the users will be able to verify the proper updates of their secret keys.
- User $u \in \mathcal{U}$, by running the *UKGen* algorithm, chooses uniformly at random a secret polynomial $f_{k,u} \in \mathbb{F}_p[X]$ of degree $= d_k - 1$ and a secret $\rho_{k,u} \leftarrow \{1, \dots, p-1\}$ value for each attribute authority k . The shares generated by the $f_{k,u}$ secret polynomial are the user u provided part of the attribute secret keys. These secret keys will be updated by authority $\mathcal{A}_k$ during the attribute key registration process. The $\rho_{k,u}$ values will be the hiding factors of the $f_{k,u}$ secret polynomials. These hiding factors are used to prevent the authorities from learning the user's secret keys.
- Attribute authority k receives the masked shares of the user u 's secret polynomial with the corresponding certificates and the user specific $s_{k,u} \cdot P$ value (encrypted by the central authority). The attribute authority k checks the certificates and decrypt the received $s_{k,u} \cdot P$ value. The masked shares of user u 's secret polynomial were calculated the following way $\rho_{k,u} \cdot f_{k,u}(\iota(a))$ where $\rho_{k,u} \leftarrow \{0, \dots, p-1\}$ are chosen uniformly at random by the user u for every authority k . The authority also receives $\rho_{k,u} \cdot P$, $\rho_{k,u} \cdot A$ and $\rho_{k,u} \cdot B$ values from user u to be able to re-randomize its secret keys. The authority k re-randomizes user u 's secret shares by running the *AKGen* algorithm to generate the $f_{k,u}^r$ polynomial and to compute the values $f_{k,u}^r(\iota(a))$. The

authority performs the following computation

$$\underbrace{\rho_{k,u} \cdot f_{k,u}(\iota(a))}_{\text{masked user's share}} \cdot B + \rho_{k,u} \cdot B \cdot \underbrace{f_{k,u}^r(\iota(a))}_{\text{authority's share}} \in G.$$

The authority k computes the masked attribute specific secret keys for the approved credentials of user u by running *Update1*

$$\frac{\rho_{k,u} \cdot f_{k,u}(\iota(a)) \cdot B + \rho_{k,u} B \cdot f_{k,u}^r(\iota(a))}{t_{k,a}} \in G.$$

The authority k computes the second part of the attribute secret keys as follows

$$X_{k,u} = s_{k,u} \cdot P - f_{k,u}^r(0) \cdot B + r_k \cdot P.$$

The authority sends the computed values to user u .

- The user updates the received values by running the *Update2* algorithm, removing the $\rho_{k,u}$ hiding factor from the shares and subtracting $f_{k,u}(0) \cdot B$ from $X_{k,u}$. These new values are the updated secret keys of user u :

$$D_{k,u,a} = \frac{f_{k,u}(\iota(a)) \cdot B + f_{k,u}^r(\iota(a)) \cdot B}{t_{k,a}} \in G$$

$$X_{k,u} = s_{k,u} \cdot P - f_{k,u}^r(0) \cdot B + r_k \cdot P - f_{k,u}(0) \cdot B$$

Encryption. To encrypt a message $M \in G_2$ with associated attribute set $\mathcal{A}_C \subseteq \mathcal{A}$, the encrypting party chooses $s \leftarrow \{0, \dots, p-1\}$ uniformly at random and computes the ciphertext as follows

$$\left(\left(\text{pk} \cdot \prod_{k \in \mathcal{K}} e(P, P)^{r_k} \right)^s \cdot M, s \cdot P, s \cdot T_{k_{CA}}, [s \cdot T_{k,a}]_{a \in \mathcal{A}_C} \right).$$

Decryption. Let $C = ((\text{pk} \cdot \prod_{k \in \mathcal{K}} e(P, P)^{r_k})^s \cdot M, s \cdot P, s \cdot T_{k_{CA}}, [s \cdot T_{k,a}]_{a \in \mathcal{A}_C})$ be a ciphertext with associated attribute set $\mathcal{A}_C$, and suppose that user u 's attribute set $\mathcal{A}_u$ satisfies $|\mathcal{A}_C \cap \mathcal{A}_u \cap \mathcal{A}_k| \geq d_k$ for all $k \in \mathcal{K}$. Then u can recover the plaintext M as follows.

1. For each $k \in \mathcal{K}$, he chooses d_k attributes $a \in \mathcal{A}_u \cap \mathcal{A}_k$, and computes

$$e(s \cdot T_{k,a}, D_{k,u,a}) = e(P, B)^{f_{k,u}(\iota(a)) \cdot s + f_{k,u}^r(\iota(a)) \cdot s}.$$

Then, using Lagrange polynomial interpolation, u computes

$$e(P, B)^{f_{k,u}(0) \cdot s + f_{k,u}^r(0) \cdot s}.$$

2. For the attribute that was provided by k_{CA} user u computes

$$e(s \cdot T_{k_{CA}}, D_{k,u}) = e(P, P)^{s \cdot s_{k_{CA},u}}.$$

3. Further on, for each $k \in \mathcal{K}$, user u can use the $X_{k,u}$ component of his secret key to compute

$$e(X_{k,u}, s \cdot P) = e(P, P)^{(s_{k,u} + r_k - f_{k,u}(0) \cdot \beta - f_{k,u}^r(0) \cdot \beta) \cdot s}.$$

4. Finally, user u computes the following product:

$$\begin{aligned} & e(P, P)^{s \cdot sk_{CA,u}} \prod_{k \in \mathcal{K}} e(P, B)^{(f_{k,u}(0) + f_{k,u}^r(0)) \cdot s} e(X_{k,u}, s \cdot P) \\ &= (P, P)^{s \cdot sk_{CA,u}} \cdot e(P, P)^{s \cdot \sum_{k \in \mathcal{K}} (s_{k,u} + r_k)} \\ &= e(P, P)^{s \cdot (\sigma + \sum_{k \in \mathcal{K}} r_k)} = \left(pk \cdot \prod_{k \in \mathcal{K}} e(P, P)^{r_k} \right)^s \end{aligned}$$

By inverting this element and multiplying the result with the first component of the ciphertext, the plaintext M can be recovered.

4.2 Adding new authorities

The user specific secrets $sk_{CA,u}$ of the central authority k_{CA} could facilitate the integration of new authorities into an existing protocol. To add a new authority k^* , the central authority replaces the old value σ with a new uniformly random value σ' , and updates each $sk_{CA,u}$ as follows:

$$sk_{CA,u} = \sigma' - \sigma - sk_{k^*,u}$$

where $sk_{k^*,u} \leftarrow \{0, \dots, p-1\}$ is the new authority specific value. The k_{CA} sends the updated central authority specific attribute secret keys

$$\frac{sk_{CA,u}}{t_{k_{CA}}} \cdot P$$

to the users. The new authority k^* can compute its secret and public keys as before and initiate the interactive attribute key generation process.

5 Security

The security of the partially registered multi-authority attribute-based encryption scheme in the selective-ID model will be proven. To start our discussion we consider the three different types of key queries as it was mentioned in the earlier security definition. The informal description of the three different types of queries:

- **Register honest key query:** The adversary is allowed to ask the challenger to generate honest keys. The challenger samples the public keys and secret keys. The adversary does not contribute to the randomness of the keys.

- **Corrupt honest key query:** The adversary is allowed to request the secret keys that are corresponding to honestly generated keys as long as there exists an honest authority from which he has less than the necessary number of secret keys to reconstruct the secret sharing polynomial. In response, the challenger looks up the corresponding secret key and replies to adversary with the secret key.
- **Register corrupted key query:** The adversary is allowed to register corrupted attribute secret keys, as long as there exists an honest authority from which he has less than the necessary number of secret keys to reconstruct the secret polynomial. The adversary contributes to the randomness of the attribute secret keys when performing the interactive key generation with the involved attribute authorities.

If the adversary is limited to the first two types of key queries, then the security proof is similar to the one in Božović et. al. [BSSV12] paper. This is the case when the user prefers not to register their own attribute secret keys and the authorities generate them for the user. In this way, our newly proposed partially registered multi-authority attribute-based encryption scheme can be transformed into the previously proposed non-registered multi-authority attribute-based encryption scheme. In the rest of the paper when discuss the security of the scheme the adversary will be permitted to issue all three types of secret key queries. If we consider an insider attack from a curious central authority, a security proof identical to the one in Božović et. al. [BSSV12] paper would work as well. From the viewpoint of this central authority, the protocol has not changed much in comparison to the previous protocol with the honest-but-curious central authority. The same security model and security proof is applicable. The same assumption is needed, i.e. the central authority cannot issue attribute secret keys queries. More details about the differences and similarities between the schemes (from the view point of the central authority) will be provided in *Appendix B*.

5.1 The interactive attribute secret key generation with a hiding factor

In the rest of this paper, we focus on our main case when an adversary is permitted to issue all types of key queries, even the register corrupted key queries, and we prove security in the selective-ID model. The authorities have to obtain the $\rho_k \cdot P$, $\rho_k \cdot A$ and $\rho_k \cdot B$ values to be able to answer secret key queries. The adversary provides these extra $\{\rho_{k,u}A\}_{k \in K}$ and $\{\rho_{k,u}B\}_{k \in K}$ values to the authorities when registering secret keys with attribute authority k .

We subsequently refer to $\mathcal{S}$ as the *Simulator*: it will simulate all attribute authorities and the central authority to adversary $\mathcal{H}$, and $\mathcal{S}$ will answer all queries for user keys made by $\mathcal{H}$. Algorithm $\mathcal{S}$ receives the following tuple for all $k \in K$ from the adversary

$$([f_{k,u}(\iota(a))]_{a \in \mathcal{A}_u} \cdot \rho_{k,u}, \rho_{k,u} \cdot P, \rho_{k,u} \cdot A, \rho_{k,u} \cdot B).$$

The tuple contains no valuable information about the masked and shared secret value $f_{k,u}(0)$ that could help $\mathcal{S}$ (which runs in probabilistic polynomial time)

break the security of the protocol and learn this $f_{k,u}(0)$ value from the given $f_{k,u}(0)\rho_{k,u}$ value. Specially, if α from $A = \alpha \cdot P$ and $\rho_{k,u} \cdot P$ are known then $\rho_{k,u} \cdot A = \alpha \cdot (\rho_{k,u} \cdot P)$ is easily computable without any help (same for β). If $f_{k,u}(0) \cdot \rho_{k,u}$ (derived through Lagrange interpolation from $f_{k,u}(\iota(a)) \cdot \rho_{k,u}$ values when the $\mathcal{S}$ has obtained at least d_k shares) and $\rho_{k,u} \cdot P$ are known and if the $\mathcal{S}$ can calculate the $f_{k,u}(0)$ value, then $\mathcal{S}$ would be able to solve a discrete logarithm problem (compute $\rho_{k,u}$), which is considered unsolvable in polynomial time. To summarize it, if the *Simulator* receives the above specified tuple

$$([f_{k,u}(\iota(a))]_{a \in A_u} \cdot \rho_{k,u}, \rho_{k,u} \cdot P, \rho_{k,u} \cdot A, \rho_{k,u} \cdot B),$$

then $\mathcal{S}$ will be able to issue secret keys for user u without violating security.

5.2 Security in the selective-ID model

The theorems proven in this section show that the protocol proposed is secure in the sense of our definition. The basic security guarantee of the protocol in question is given by Theorem 1. Our proof builds on the analysis of Božović et al. [BSSV12]. It is worth noting that the reduction to a D-BDH adversary $\mathcal{S}$ in the proof is still tight: Essentially, the advantage of the adversary $\mathcal{H}$ violating security in the selective-ID model is only halved at the cost of simulating the attribute authorities k and the central authority k_{CA} .

Theorem 1. *Suppose there exists a probabilistic polynomial time adversary $\mathcal{H}$ against the protocol in Section 4.1 having a non-negligible advantage in the game in Definition 3. Then there is a probabilistic polynomial time algorithm $\mathcal{S}$ having a non-negligible advantage in solving the D-BDH-problem.*

Proof. As explained in Section 3.1, the input of the D-BDH adversary $\mathcal{S}$ is a tuple

$$(P, \alpha P, \beta P, \gamma P, e(P, P)^{\delta \cdot \alpha \beta \gamma + (1-\delta) \cdot \eta}) \quad (1)$$

with $\delta \leftarrow \{0, 1\}$ being chosen uniformly random. To find δ , the algorithm $\mathcal{S}$ runs a simulation of $\mathcal{H}$. More specifically, $\mathcal{S}$ mimics the individual phases of the game in Definition 3 as follows:

Setup. The simulator uses the threshold values of the attribute authorities along with the attribute sets defined by $\mathcal{H}$. For authorities that have been compromised, the simulator follows the original protocol specification exactly, ensuring that the history of such an authority (revealed to $\mathcal{H}$) follows the same distribution as in the game in Definition 3. For honest attribute authorities, the simulator still emulates their behavior, but with a modification: rather than computing the public key of an uncorrupted authority k as $(e(P, P)^{r_k}, [t_{k,a} \cdot P]_{a \in A_k})$, it instead uses $(e(P, P)^{r_k}, [t_{k,a} \cdot Q]_{a \in A_k})$, where:

$$Q := \begin{cases} P, & \text{if } a \in A_C \\ \beta P, & \text{if } a \notin A_C \end{cases}$$

Here, βP is derived from the D-BDH challenge instance. In other words, for attributes $a \in \mathcal{A}_k \setminus \mathcal{A}_C$ handled by honest authorities, the random value $t_{k,a}$ is multiplied with the point βP instead of P . Since G_1 is of prime order, with overwhelming probability βP generates G_1 and for $\mathcal{H}$ the distribution of the public keys does not change compared to the game in Definition 3. Reflecting this modification of public keys, the computation of the polynomials $f_{k,u}^r$ by honest authorities will also be modified, and the simulator $\mathcal{S}$ will define the polynomials $f_{k,u}^r$ implicitly when answering secret key queries as detailed below. When simulating the central authority k_{CA} , the simulator follows the steps of the original protocol, with the following exceptions:

- The value pk in the public system parameters is computed as

$$\text{pk} := e(\alpha P, \beta P) \quad (2)$$

where αP and βP are part of the D-BDH challenge. For the adversary $\mathcal{H}$, the usage of this modified pk -value instead of $e(P, P)^\sigma$ makes no difference. Because of G_2 being of prime order, with overwhelming probability $\text{pk} = e(P, P)^{\alpha\beta}$ is a uniformly distributed element in G_2 , just like the original value $e(P, P)^\sigma$. The additional information on σ that is potentially available to $\mathcal{H}$ is

1. αP and βP values, $e(\alpha P, \beta P) = e(P, P)^\sigma$,
2. $s_{k,u} \cdot P$ -values of corrupted authorities,
3. $X_{k,u} = s_{k,u} \cdot P - f_{k,u}^r(0) \cdot B + r_k \cdot P - f_{k,u}(0) \cdot B$ -values obtained from secret user key queries,
4. $[\frac{s_{k_{CA},u}}{t_{k_{CA}}} \cdot P]_{u \in \mathcal{U}}$, the central authority specific attribute secret keys of the users.

By assumption, for each $u \in \mathcal{U}$, at least one authority $\hat{k}(u)$ is uncorrupted. The elements listed above do not reveal any information on σ . Even with the knowledge of the $S_{k,u}$ -values of all corrupted authorities and with the $[\frac{s_{k_{CA},u}}{t_{k_{CA}}} \cdot P]_{u \in \mathcal{U}}$ -values, each value of σ remains equally likely, as for each $u \in \mathcal{U}$ the equation

$$\sigma = \sum_{k \in \mathcal{K} \cup \{k_{CA}\}} s_{k,u}$$

contains at least one unknown random value $s_{\hat{k}(u),u}$. The only potentially available information on $s_{\hat{k}(u),u}$ are the value $X_{\hat{k}(u),u}$ and $e(P, P)^{s_{\hat{k}(u),u}}$ obtained from a secret user key query. However, due to the subtraction of the random value $f_{k,u}^r(0) \cdot P$, each $X_{k,u}$ is an independent random value, containing no information on $s_{k,u}$ or σ . To derive $s_{\hat{k}(u),u}$ from $e(P, P)^{s_{\hat{k}(u),u}}$ is equivalent to solving a discrete logarithm problem in G_2 what was considered a hard problem.

- The simulator chooses the $s_{k_{CA},u}$ ($u \in \mathcal{U}$) and the $s_{k,u}$ -values of corrupted authorities uniformly at random. For honest authorities, the $s_{k,u}$ -values will be determined later as needed.

Secret key queries. We can w.l.o.g. assume that $\mathcal{H}$ does not query secret user keys from corrupted attribute authorities, as $\mathcal{H}$ can compute such user keys itself. For uncorrupted attribute authorities, the simulator $\mathcal{S}$ must be able to answer secret key queries from $\mathcal{H}$. At first, let us explain the basic idea behind the secret key queries and the related hard problems. Here we focus on the part of the secret keys that were originally generated and issued by authority k .

sets	$\mathcal{A}_C \cap \mathcal{A}_k \cap \mathcal{A}_u$	$\mathcal{A}_C \cap \mathcal{A}_k \setminus \mathcal{A}_u$	$\mathcal{A}_k \cap \mathcal{A}_u \setminus \mathcal{A}_C$
attributes	$t_{k,a}P$	$t_{k,a}P$	$t_{k,a}\beta P$
encryption	$t_{k,a}\gamma P$	$t_{k,a}\gamma P$	Hard problem 1
user u 's sk	$\frac{\xi_{k,u}}{t_{k,a}}\beta P$	Hard problem 2	$\frac{f_{k,u}^r(a)}{t_{k,a}}P$

Table 1. Security proof summary when $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| < d_k$ and authority k is honest. We assume $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| = d_k - 1$. If $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| < d_k - 1$ then the adversary can ask for further secret user keys. We fix the value of the $f_{k,u}^r$ polynomial at 0, i.e. $f_{k,u}^r(0) := \alpha P + \xi_k P$.

sets	$\mathcal{A}_C \cap \mathcal{A}_k \cap \mathcal{A}_u$	$\mathcal{A}_C \cap \mathcal{A}_k \setminus \mathcal{A}_u$	$\mathcal{A}_k \cap \mathcal{A}_u \setminus \mathcal{A}_C$
attributes	$t_{k,a}P$	$t_{k,a}P$	$t_{k,a}\beta P$
encryption	$t_{k,a}\gamma P$	$t_{k,a}\gamma P$	Hard problem 1
user u 's sk	$\frac{\xi_{k,u}}{t_{k,a}}\beta P$ or $\frac{f_{k,u}^r(a)}{t_{k,a}}\beta P$ ¹	$\frac{f_{k,u}^r(a)}{t_{k,a}}\beta P$	$\frac{f_{k,u}^r(a)}{t_{k,a}}P$

Table 2. Security proof summary when $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| \geq d_k$ and authority k is honest. In this second case we do not fix the value of the $f_{k,u}^r$ polynomial at 0. It will be computed from the d_k random shares $(\xi_{k,u})$.

We differentiate the attributes into three different sets based on their involvement in the encryption process (involved: 1st and 2nd columns, not involved 3rd column) and the adversary's capability to request secret keys for the attributes (allowed to request: 1st and 3rd columns, not allowed 2nd column). As shown in the table, there are cases when we created hard problems (see in Table 1 and in Table 2). In Hard Problem 1, the simulator cannot produce the encryption of the message using the given attributes, as it cannot compute $\beta\gamma P$ to generate the $t_{k,a} \cdot \beta\gamma P$ values. However, these computations are not necessary because these attributes are not included in the challenge set. In Hard Problem 2, the simulator cannot answer these secret key queries because it is unable to compute $\alpha\beta P$ to issue the secret keys. However, this is not required since the user does not have the corresponding credentials to request these secret keys.

We are going to identify four different cases of secret queries. The last two cases

¹ The first d_k values, $\xi_{k,u}$, are chosen uniformly at random, the remaining values are computed by using Lagrange interpolation.

are similar to the cases in [BSSV12] paper. The new scheme can be considered as an extension of the previous scheme. These cases are related to **corrupt honest key query** after the register honest key query. In these cases, the challenger generates the attribute secret keys by running the $CKGen$, $UKGen$, $AKGen$, and $Update1$, $Update2$ algorithms, or it may simplify the procedure by running only the $CKGen$ and $AKGen$ algorithms (as using the other two algorithms does not provide an additional level of security, as mentioned earlier). The first two cases are related to **register corrupted key query**. When the adversary issues these types of queries, all previously defined algorithms will be run: the $UKGen$, $Update2$ will be used by the adversary, while the $CKGen$, $AKGen$, $Update1$ will be run by the challenger. We do not have a separate case for **register honest key query** since it will not have an output to share with the adversary. The four distinct cases of secret key queries are:

1. $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| < d_k$ and there has not been a previous secret key query for user u to an authority $k' \neq k$ with $|\mathcal{A}_u \cap \mathcal{A}_{k'} \cap \mathcal{A}_C| < d_{k'}$ and the adversary issues register corrupt key query: W.l.o.g., we may assume $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| = d_k - 1$ (otherwise, we can modify $\mathcal{H}$ to ask for further secret user keys which will be ignored). The simulator implicitly defines $f_{k,u}^r$ by specifying the values of $f_{k,u}^r$ at d_k points (the d_k -th value is related to the D-BDH challenge). Namely, the simulator chooses uniformly at random $\xi_{k,u,a} \in \mathbb{F}_p$ for all $a \in \mathcal{A}_k \cap \mathcal{A}_C$, a random value $\hat{\xi}_{k,u} \in \mathbb{F}_p$, and imposes

$$\begin{aligned} f_{k,u}^r(\iota(a)) &= \beta \cdot \xi_{k,u,a} \quad \text{for all } a \in \mathcal{A}_k \cap \mathcal{A}_C \\ \text{and} \\ f_{k,u}^r(0) &= \beta \cdot (\alpha + \hat{\xi}_{k,u}) \end{aligned}$$

with αP , βP being part of the D-BDH challenge and specified at the beginning of the protocol. With overwhelming probability $\beta \neq 0$ and $f_{k,u}^r$ follows the same distribution as in the original protocol. Since all the shares (sent by the adversary) are multiplied by an unknown $\rho_{k,u}$ chosen by the adversary we perform all the computation multiplied by $\rho_{k,u} \cdot P$. We need to obtain the two additional values $\rho_{k,u} \cdot A$ and $\rho_{k,u} \cdot B$ from the adversary. We use $\rho_{k,u} \cdot \alpha$ and $\rho_{k,u} \cdot \beta$ instead of α and β during computation of the secret keys:

$$\begin{aligned} \rho_{k,u} \cdot f_{k,u}^r(\iota(a)) &= (\beta \cdot \rho_{k,u}) \cdot \xi_{k,u,a} \quad \text{for all } a \in \mathcal{A}_k \cap \mathcal{A}_C \\ \text{and} \\ \rho_{k,u} \cdot f_{k,u}^r(0) &= \beta \cdot [(\alpha \cdot \rho_{k,u}) + \hat{\xi}_{k,u} \cdot \rho_{k,u}] \end{aligned}$$

$\mathcal{S}$ can use $\rho_{k,u}$ -times the values αP , βP from the D-BDH challenge to extract the requested secret key $(X_{k,u}, D_{k,u,a})$ for user $u \in \mathcal{U}$ and attribute $a \in \mathcal{A}_k \cap \mathcal{A}_u$:

– For $a \in \mathcal{A}_C$,

$$D'_{k,u,a} = \frac{\rho_{k,u} \cdot f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot P = \frac{\xi_{k,u,a}}{t_{k,a}} \cdot \rho_{k,u} \cdot B$$

- Because of

$$\frac{\rho_{k,u} \cdot f_{k,u}^r(0)}{t_{k,a} \cdot \beta} \cdot P = \frac{\rho_{k,u} \cdot A + \hat{\xi}_{k,u} \cdot \rho_{k,u} \cdot P}{t_{k,a}}$$

the simulator $\mathcal{S}$ can compute the d_k points

$$\frac{\rho_{k,u} \cdot f_{k,u}^r(0)}{t_{k,a} \cdot \beta} \cdot P, \underbrace{\left[\frac{\rho_{k,u} \cdot f_{k,u}^r(\iota(a))}{t_{k,a} \cdot \beta} \cdot P \right]}_{\rho_{k,u} \xi_{k,u,a} / t_{k,a}} \Big]_{a \in \mathcal{A}_k \cap \mathcal{A}_C}$$

and then use Lagrange interpolation to derive

$$D'_{k,u,a} = \frac{\rho_{k,u} \cdot f_{k,u}^r(\iota(a))}{t_{k,a} \cdot \beta} \cdot P, \text{ for } a \notin \mathcal{A}_C$$

- The simulator updates the secret key components $(\rho_{k,u} \cdot f_{k,u}(\iota(a)))$ that were sent by the adversary (for user u) during the key registration process. The *Simulator* sends the following values for user u 's secret key

$$D_{k,u,a}^* := \begin{cases} D'_{k,u,a} + \frac{\rho_{k,u} f_{k,u}(\iota(a))}{t_{k,a}} \cdot B, & \text{if } a \in \mathcal{A}_C \\ D'_{k,u,a} + \frac{\rho_{k,u} f_{k,u}(\iota(a))}{t_{k,a}} \cdot P, & \text{if } a \notin \mathcal{A}_C \end{cases}$$

When the adversary receives the user u 's attribute secret key the adversary remove the $\rho_{k,u}$ values.

The attribute secret keys are

$$D_{k,u,a} := \begin{cases} \frac{\xi_{k,u,a} + f_{k,u}(\iota(a))}{t_{k,a}} \cdot B, & \text{if } a \in \mathcal{A}_C \\ \frac{f_{k,u}^r(\iota(a)) + f_{k,u}(\iota(a))}{t_{k,a}} \cdot P, & \text{if } a \notin \mathcal{A}_C \end{cases}$$

- Finally, the simulator computes

$$X_{k,u}^* := r_k \cdot P - \hat{\xi}_{k,u} \cdot \beta P - \left(\sum_{\kappa \in \mathcal{K} \setminus \{k\}} s_{\kappa,u} \right) \cdot P,$$

choosing, for user u , all $S_{\kappa,u}$ ($\kappa \in \mathcal{K} \setminus \{k\}$) that have not been fixed already, as $S_{\kappa,u} := s_{\kappa,u} \cdot P$ with a uniformly at random chosen $s_{\kappa,u}$. With the modified value of pk in 2, this choice of $X_{k,u}$ implicitly fixes $s_{k,u} := \alpha\beta - \sum_{\kappa \in \mathcal{K} \setminus \{k\}} s_{\kappa,u} - s_{k_{CA},u}$. The simulator sends $X_{k,u}$ to the adversary.

- When the adversary receives the $X_{k,u}^*$ value he updates it, by subtracting the secret $f_{k,u}(0) \cdot \beta P$ value from it.

$$X_{k,u} := r_k \cdot P - f_{k,u}(0) \cdot \beta P - \hat{\xi}_{k,u} \cdot \beta P - \left(\sum_{\kappa \in \mathcal{K} \setminus \{k\}} s_{\kappa,u} \right) \cdot P$$

2. $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| \geq d_k$ or there has been a previous secret key query for user u to an authority $k' \neq k$ with

$$|\mathcal{A}_u \cap \mathcal{A}_{k'} \cap \mathcal{A}_C| < d_{k'}$$

and the adversary issues a register corrupt key query: In this case, the simulator chooses a random polynomial $f_{k,u}^r \in \mathbb{F}_p[X]$ of degree $= d_k - 1$ and implicitly defines $g_{k,u} := \beta \cdot (f_{k,u} + f_{k,u}^r)$ (with βP being part of the D-BDH challenge). Note that, with overwhelming probability $\beta \neq 0$ and $f_{k,u}^r$ follows the same distribution as in the original protocol. Using the value βP from the D-BDH challenge, $\mathcal{S}$ computes the respective secret key $(X_{k,u}^*, D_{k,u,a}^*)$ for user $u \in \mathcal{U}$ and attribute $a \in \mathcal{A}_k \cap \mathcal{A}_u$ as follows:

$$D_{k,u,a}^* := \begin{cases} \frac{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot \rho_{k,u} B, & \text{if } a \in \mathcal{A}_C \\ \frac{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot \rho_{k,u} P, & \text{if } a \notin \mathcal{A}_C \end{cases}$$

$$X_{k,u}^* := r_k \cdot P - f_{k,u}^r(0) \cdot \beta P + S_{k,u}$$

At this point, if the value $S_{k,u}$ is not already fixed through a previous secret key query (see above), it is chosen as $S_{k,u} := s_{k,u} \cdot P$ with $s_{k,u}$ chosen uniformly at random. The adversary updates the keys as before:

$$D_{k,u,a} := \begin{cases} \frac{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot B, & \text{if } a \in \mathcal{A}_C \\ \frac{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot P, & \text{if } a \notin \mathcal{A}_C \end{cases}$$

$$X_{k,u} := r_k \cdot P - f_{k,u}(0) \cdot \beta P - f_{k,u}^r(0) \cdot \beta P + S_{k,u}.$$

3. $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| < d_k$ and there has not been a previous secret key query for user u to an authority $k' \neq k$ with $|\mathcal{A}_u \cap \mathcal{A}_{k'} \cap \mathcal{A}_C| < d_{k'}$, and the adversary issues corrupt honest key query after the register honest key query: W.l.o.g., we may assume $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| = d_k - 1$. The simulator implicitly defines $f_{k,u}^r$ by specifying the values of $f_{k,u}^r$ at d_k points. Namely, the simulator chooses uniformly at random $\xi_{k,u,a} \in \mathbb{F}_p$ for all $a \in \mathcal{A}_k \cap \mathcal{A}_C$, a random value $\hat{\xi}_{k,u} \in \mathbb{F}_p$, and imposes

$$\begin{aligned} f_{k,u}^r(\iota(a)) &= \beta \cdot \xi_{k,u,a} \quad \text{for all } a \in \mathcal{A}_k \cap \mathcal{A}_C \\ \text{and} \\ f_{k,u}^r(0) &= \beta \cdot (\alpha + \hat{\xi}_{k,u}) \end{aligned}$$

with $\alpha P, \beta P$ being part of the D-BDH challenge, and specified at the beginning of the protocol. With overwhelming probability $\beta \neq 0$ and $f_{k,u}^r$ follows the same distribution as in the original protocol. $\mathcal{S}$ can use $\alpha P, \beta P$ from the D-BDH challenge to extract the requested secret key $(X_{k,u}, D_{k,u,a})$ for user $u \in \mathcal{U}$ and attribute $a \in \mathcal{A}_k \cap \mathcal{A}_u$:

- For $a \in \mathcal{A}_C$,

$$D_{k,u,a} = \frac{f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot P = \frac{\xi_{k,u,a}}{t_{k,a}} \cdot B.$$

- Because of

$$\frac{f_{k,u}^r(0)}{t_{k,a} \cdot \beta} \cdot P = \frac{A + \hat{\xi}_{k,u} \cdot P}{t_{k,a}}$$

the simulator $\mathcal{S}$ can compute the d_k points

$$\frac{f_{k,u}^r(0)}{t_{k,a} \cdot \beta} \cdot P, \underbrace{\left[\frac{f_{k,u}^r(\iota(a))}{t_{k,a} \cdot \beta} \cdot P \right]}_{\xi_{k,u,a}/t_{k,a}} \Big]_{a \in \mathcal{A}_k \cap \mathcal{A}_C}$$

and then use Lagrange interpolation to derive

$$D_{k,u,a} = \frac{f_{k,u}^r(\iota(a))}{t_{k,a} \cdot \beta} \cdot P, \text{ for } a \notin \mathcal{A}_C.$$

- Finally, the simulator computes

$$X_{k,u} := r_k \cdot P - \hat{\xi}_{k,u} \cdot \beta P - \left(\sum_{\kappa \in \mathcal{K} \setminus \{k\}} s_{\kappa,u} \right) \cdot P,$$

choosing, for the user u , all $S_{\kappa,u}$ ($\kappa \in \mathcal{K} \setminus \{k\}$), that have not been fixed already, as $S_{\kappa,u} := s_{\kappa,u} \cdot P$ with a uniformly at random chosen $s_{\kappa,u}$. With the modified value of pk in 2, this choice of $X_{k,u}$ implicitly fixes

$$s_{k,u} := \alpha\beta - \sum_{\kappa \in \mathcal{K} \setminus \{k\}} s_{\kappa,u} - s_{k_{CA},u}.$$

4. $|\mathcal{A}_u \cap \mathcal{A}_k \cap \mathcal{A}_C| \geq d_k$ or there has been a previous secret key query for user u to an authority $k' \neq k$ with $|\mathcal{A}_u \cap \mathcal{A}_{k'} \cap \mathcal{A}_C| < d_{k'}$ and the adversary issues a corrupt honest key query after the register honest key query: In this case, the simulator chooses a random polynomial $f_{k,u}^r \in \mathbb{F}_p[X]$ of degree and implicitly defines $g_{k,u} := \beta \cdot f_{k,u}^r$ (with βP being part of the D-BDH challenge). Note that, with overwhelming probability $\beta \neq 0$ and $f_{k,u}^r$ follows the same distribution as in the original protocol. Using the value βP from the D-BDH challenge, $\mathcal{S}$ computes the respective secret key $(X_{k,u}, D_{k,u,a})$ for user $u \in \mathcal{U}$ and attribute $a \in \mathcal{A}_k \cap \mathcal{A}_u$ as follows:

$$D_{k,u,a} := \begin{cases} \frac{f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot B, & \text{if } a \in \mathcal{A}_C \\ \frac{f_{k,u}^r(\iota(a))}{t_{k,a}} \cdot P, & \text{if } a \notin \mathcal{A}_C \end{cases}$$

$$X_{k,u} := r_k \cdot P - f_{k,u}^r(0) \cdot \beta P + S_{k,u}$$

At this point, the value $S_{k,u}$, if not fixed already through a previous secret key query (see above), is chosen as $S_{k,u} := s_{k,u} \cdot P$ with a uniformly at random chosen $s_{k,u}$.

Challenge. Let $M_0, M_1 \in G_2$ be the challenge messages selected by the adversary $\mathcal{H}$, and let δ be the value to be found by the D-BDH adversary $\mathcal{S}$. Using a fair binary coin $\mu \leftarrow \{0, 1\}$ and the last two components of the D-BDH challenge and the message M_μ , the simulator hands to $\mathcal{H}$ the following challenge ciphertext

$$\left(C' \cdot e(\gamma P, P)^{\sum_{k \in \mathcal{K}} r_k} \cdot M_\mu, \gamma P, t_{k_{CA}} \gamma P, [t_{k,a} \gamma P]_{a \in \mathcal{A}_C} \right), \quad (3)$$

where $C' = e(P, P)^{\delta \cdot \alpha \beta \gamma + (1-\delta) \cdot \eta}$.

We consider both possible cases $\delta = 0$ and $\delta = 1$:

$\delta = 0$: In this case, the $e(P, P)^{\delta \cdot \alpha \beta \gamma + (1-\delta) \cdot \eta} = e(P, P)^\eta$ with a uniformly at random chosen $\eta \leftarrow \{0, \dots, p-1\}$. The challenge ciphertext contains no information on M_μ .

$\delta = 1$: If we choose $\mathbf{pk} = e(\alpha P, \beta P)$ and γ as the random encryption exponent, then the challenge ciphertext (3) is a valid encryption of the given message M_μ , with $C' = e(P, P)^{\alpha \beta \gamma}$.

Further secret key queries. The simulator proceeds with further secret key queries exactly as it did before the challenge phase, maintaining consistency with already answered secret key queries.

Guess. Denote by μ' the output of $\mathcal{H}$. The output of the simulator $\mathcal{S}$ is given by

$$\delta' := \begin{cases} 1, & \text{if } \mu = \mu' \\ 0, & \text{if } \mu \neq \mu' \end{cases}$$

In other words, $\mathcal{S}$ considers the last component of the D-BDH challenge to be $e(P, P)^{\alpha \beta \gamma}$ whenever $\mathcal{H}$ correctly identifies M_μ . As in case of $\delta = 0$ the challenge ciphertext contains no information about μ , the adversary $\mathcal{H}$'s probability of correctly identifying μ is $1/2$. Consequently, the probability that $\mathcal{S}$ returns a correct guess for δ in this case is also $1/2$:

$$\Pr(\delta' = \delta \mid \delta = 0) = \frac{1}{2}. \quad (4)$$

If $\delta = 1$, the adversary $\mathcal{H}$ faces a valid encryption of M_μ , and we obtain

$$\Pr(\delta' = \delta \mid \delta = 1) = \Pr(\mu' = \mu \mid \delta = 1) = \frac{1}{2} + \text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell). \quad (5)$$

Combining (4) and (5), we can compute $\mathcal{S}$'s advantage in solving the D-BDH challenge:

$$\begin{aligned} \text{Adv}_{\mathcal{S}}^{\text{bdh}}(\ell) &= \Pr(\delta' = \delta) - \frac{1}{2} \\ &= \frac{1}{2} \cdot (\Pr(\delta' = \delta \mid \delta = 0) + \Pr(\delta' = \delta \mid \delta = 1)) - \frac{1}{2} \\ &= \frac{1}{2} \cdot \left(\frac{1}{2} + \frac{1}{2} + \text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell) \right) - \frac{1}{2} \\ &= \frac{1}{2} \cdot \text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell). \end{aligned}$$

Since the D-BDH problem is assumed to be a hard, no PPT adversary can break it with more than $\text{Adv}_{\mathcal{S}}^{\text{bdh}}(\ell) < \text{negl}(\ell)$ probability. This implies that the advantage of the adversary attacking the encryption scheme is also negligible ($\text{Adv}_{\mathcal{H}}^{\text{sid}}(\ell)$).

5.3 Verification of the secret key updates

If the central authority shares the $e(P, P)^{s_{k,u}}$ values with user u for all attribute authorities $k \in \mathcal{K}$ during their initial communication round, then user u can check whether authority k properly updates his secret keys with a simple computation. If user u has enough (at least d_k) attribute decryption keys then it performs the following computation:

For each $k \in \mathcal{K}$ authority and attribute $a \in \mathcal{A}_u \cap \mathcal{A}_k$, user u computes

$$e(T_{k,a}, D_{k,u,a}) = e(P, B)^{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}.$$

Then, using Lagrange polynomial interpolation user u computes

$$e(P, B)^{f_{k,u}(0) + f_{k,u}^r(0)}.$$

User u checks for each $k \in \mathcal{K}$ authority whether the following equation holds

$$e(X_{k,u}, P) \cdot e(P, B)^{f_{k,u}(0) + f_{k,u}^r(0)} = e(P, P)^{s_{k,u}} \cdot e(P, P)^{r_k},$$

where $X_{k,u} = s_{k,u} \cdot P - f_{k,u}(0) \cdot B - f_{k,u}^r(0) \cdot B + r_k \cdot P$, and $f_{k,u}(0)$ is known by user u but unknown to the attribute authority k . The authority is able to calculate only the $s_{k,u} \cdot P - f_{k,u}^r(0) \cdot B + r_k \cdot P$ part of the value. If the equation is not valid for any authority $k \in \mathcal{K}$, then user u rejects the updated keys from authority k .

The hardness of inverting a bilinear pairing (e.g., computing the value of $S_{k,u}$ from $e(P, P)^{s_{k,u}}$) is required here, but it was a requirement also in the previous schemes. The hardness of the bilinear pairing inversion problem is discussed in several earlier works [Ver04, GHV08].

5.4 Attack on the central authority's secret key

This section addresses an attack on the central authority's secret key. This attack is aimed at calculating the central authority's secret key. If the adversary corrupts every single authority and obtains all $s_{k,u}P$ and all r_kP values, our scheme remains provably secure as long as the central authority is honest. The adversary is still unable to compute the central authority's secret key $\sigma \cdot P$ because

$$\sigma \cdot P = s_{k_{CA},u} \cdot P + \sum_{k \in \mathcal{K}} s_{k,u} \cdot P$$

and the adversary has no information (as already shown in section 5.2) about the $s_{k_{CA},u} \cdot P$ additive term. The only publicly available information about $s_{k_{CA},u} \cdot P$ term is the corresponding attribute value $t_{k_{CA}} \cdot P$, which is a random value. If the adversary is permitted to issue secret key queries and receives a user specific central authority issued attribute secret key $s_{k_{CA},u}/t_{k_{CA}} \cdot P$, then the adversary is able to decrypt messages but still cannot calculate the $\sigma \cdot P$ value or impersonate the central authority to issue central authority type of new secret keys for new users. The value $s_{k_{CA},u} \cdot P$ is still unknown (the only available information is $s_{k_{CA},u}/t_{k_{CA}} \cdot P$), which implies that σ remains a random value.

6 Conclusion

We presented an extension of the multi-authority attribute-based encryption scheme from Božović et al. [BSSV12]. Through the discussed modifications, we enhanced the security of their scheme by allowing users to actively participate in the attribute key generation process. After the joint key generation process, only the users are able to derive their final updated attribute secret keys. While the authorities still participate in the attribute key generation, they no longer have access to the updated versions of the attribute secret keys. When a new user joins the our system, there is no need to update the public or secret key pairs of existing users, nor the authorities' keys. We proved the security of our new scheme in the selective-ID model, assuming an honest-but-curious central authority. The performed modifications make our scheme closer to the fully registered multi-authority attribute-based encryption.

Acknowledgements

The first author is grateful for the support of the National Research, Development and Innovation Office within the framework of the Thematic Excellence Program 2021 - National Research Subprogramme: “Artificial intelligence, large networks, data security: mathematical foundation and applications”.

References

- AT24. Nuttapong Attrapadung and Junichi Tomida. A modular approach to registered abe for unbounded predicates. In *Annual International Cryptology Conference*, pages 280–316. Springer, 2024.
- BF03. Dan Boneh and Matthew Franklin. Identity-based encryption from the weil pairing. *SIAM journal on computing*, 32(3):586–615, 2003.
- BSSV12. Vladimir Božović, Daniel Socek, Rainer Steinwandt, and Viktória I Villányi. Multi-authority attribute-based encryption with honest-but-curious central authority. *International Journal of Computer Mathematics*, 89(3):268–283, 2012.
- CC09. Melissa Chase and Sherman SM Chow. Improving privacy and security in multi-authority attribute-based encryption. In *Proceedings of the 16th ACM conference on Computer and communications security*, pages 121–130, 2009.
- Cha07. Melissa Chase. Multi-authority attribute based encryption. In *Theory of Cryptography: 4th Theory of Cryptography Conference, TCC 2007, Amsterdam, The Netherlands, February 21-24, 2007. Proceedings 4*, pages 515–534. Springer, 2007.
- CHH⁺24. Sohto Chiku, Keisuke Hara, Keitaro Hashimoto, Toi Tomita, and Junji Shikata. How to apply fujisaki-okamoto transformation to registration-based encryption. In *International Conference on Cryptology and Network Security*, pages 145–165. Springer, 2024.

- CHK03. Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In *Advances in Cryptology—EUROCRYPT 2003: International Conference on the Theory and Applications of Cryptographic Techniques, Warsaw, Poland, May 4–8, 2003 Proceedings 22*, pages 255–271. Springer, 2003.
- FKdP23. Dario Fiore, Dimitris Kolonelos, and Paola de Perthuis. Cuckoo commitments: Registration-based encryption and key-value map commitments for large spaces. *Cryptology ePrint Archive*, 2023.
- GHMR18. Sanjam Garg, Mohammad Hajiabadi, Mohammad Mahmoody, and Ahmadreza Rahimi. Registration-based encryption: removing private-key generator from ibe. In *Theory of Cryptography: 16th International Conference, TCC 2018, Panaji, India, November 11–14, 2018, Proceedings, Part I 16*, pages 689–718. Springer, 2018.
- GHV08. S. Galbraith, F. Hess, and F. Vercauteren. Aspects of pairing inversion. *IEEE Transactions on Information Theory*, 54(12):5719–5728, 2008.
- GKMR23. Noemi Glaeser, Dimitris Kolonelos, Giulio Malavolta, and Ahmadreza Rahimi. Efficient registration-based encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 1065–1079, 2023.
- GLWW24. Rachit Garg, George Lu, Brent Waters, and David J Wu. Reducing the crs size in registered abe systems. In *Annual International Cryptology Conference*, pages 143–177. Springer, 2024.
- GV20. Rishab Goyal and Satyanarayana Vusirikala. Verifiable registration-based encryption. In *Advances in Cryptology—CRYPTO 2020: 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17–21, 2020, Proceedings, Part I 40*, pages 621–651. Springer, 2020.
- HLWW23. Susan Hohenberger, George Lu, Brent Waters, and David J Wu. Registered attribute-based encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 511–542. Springer, 2023.
- LW11. Allison B. Lewko and Brent Waters. Decentralizing attribute-based encryption. In Kenneth G. Paterson, editor, *Advances in Cryptology - EUROCRYPT 2011 - 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, May 15–19, 2011. Proceedings*, volume 6632 of *Lecture Notes in Computer Science*, pages 568–588. Springer, 2011.
- Sha85. Adi Shamir. Identity-based cryptosystems and signature schemes. In *Advances in Cryptology: Proceedings of CRYPTO 84 4*, pages 47–53. Springer, 1985.
- SW05. Amit Sahai and Brent Waters. Fuzzy identity-based encryption. In *Advances in Cryptology—EUROCRYPT 2005: 24th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, May 22–26, 2005. Proceedings 24*, pages 457–473. Springer, 2005.
- Ver04. Eric R Verheul. Evidence that xtr is more secure than supersingular elliptic curve cryptosystems. *Journal of Cryptology*, 17:277–296, 2004.
- ZZGQ23. Ziqi Zhu, Kai Zhang, Junqing Gong, and Haifeng Qian. Registered abe via predicate encodings. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 66–97. Springer, 2023.

A Malicious Key Curator and Trust in Registered Attribute-Based Encryption

The authors of [HLWW23] mentioned a weak point of their scheme's key generation process. The structured common reference string (CRS) must be sampled by a trusted party (or using an MPC protocol). They claim this is the only trusted component in the system. First, let us examine this weak point and consider the problem of a compromised CRS generation. The generation of CRS should be fully secure since the security of the scheme relies heavily on the secrecy of the values used during the CRS generation. Any entity with access to the secret values g_1^α (even without knowing α) when the public key is $Z = e(g_1, g_1)^\alpha$ can decrypt any ciphertext. It is trivial to break the security of scheme in this case by using only the first two elements of the ciphertext:

$$(C_1 = \mu Z^s = \mu e(g_1, g_1)^{\alpha s}, \quad C_2 = g_1^s)$$

When both g_1^α and g_1^s are known, we can use the bilinear pairing to compute

$$e(g_1, g_1)^{\alpha s} = e(g_1^\alpha, g_1^s).$$

The message can be recovered by inverting and multiplying this element by the first part of the ciphertext

$$\mu = \frac{C_1}{e(g_1^\alpha, g_1^s)}.$$

We claim there is another problematic part in the key generation process. A malicious key curator and a user (owner of the secret key) can collude to decrypt any messages. Even the key curator alone could perform this attack without any help from a user. The problematic part is the key aggregation process. The key curator aggregates the $T_j = \text{pk}_j$ public keys and the $U_{j,w}$ attribute-specific components into a compact master public key mpk . The following simple aggregation mechanisms were applied:

$$\hat{T} = \prod_{j \in [L]} T_j \quad \text{and} \quad \hat{U}_w = \prod_{j \in [L]: w \notin S_j} U_{j,w}.$$

The public keys of all user were included in the $\hat{T}$ product. The attribute-specific components $U_{j,w}$ of a user were included in the $\hat{U}_w$ product when the user j does not have a secret key for an attribute w . The aggregated public key components $\hat{T}$, $\hat{U}_w$ will be used for encryption. The master public key (mpk) is defined as

$$\text{mpk} = \left(G, g_1, h, Z, \hat{T}, \{\hat{U}_w\}_{w \in U} \right).$$

The slot-specific component, that is a part of the user's helper secret key (hsk), is a similarly aggregated value. The attribute-specific aggregated slot keys $\hat{W}_{i,w}$ for each $i \in [L]$ are computed as:

$$\hat{W}_{i,w} = \prod_{j \neq i: w \notin S_j} W_{i,j,w}$$

The $\widehat{W}_{i,j,w}$ were included in the $\widehat{W}_{i,w}$ product when the user does not have a secret key for the attribute w . The helper secret keys hsk_i are defined as:

$$\text{hsk}_i = \left(\text{mpk}, i, S_i, A_i, B_i, \widehat{V}_i, \{\widehat{W}_{i,w}\}_{w \in U} \right).$$

The key curator can perform the following modification on the computation of the mpk and hsk_i :

- Modification on the mpk
 - ✓ Include a public key $pk_j = T_j = g^{r_j}$ in the $\hat{T}$ product.
 - ✓ Do not include any corresponding $U_{j,w}$ attribute-specific values in the $\hat{U}_w$ product.
- Modification on the hsk_i
 - ✓ Do not include any corresponding attribute-specific slot keys $\hat{W}_{i,j,w}$ in the $\hat{W}_{i,w}$ product.

The original decryption algorithm will work after these modifications are applied by a malicious key curator. The key curator will be able to decrypt any messages independently of the access structure if he has access to the r_j secret key (which could have been chosen by itself). The basic idea is that the key curator assumes in the key aggregation process the attributes of user j with public key pk_j satisfy every possible access policy in the system. The owner of those keys will be able to decrypt any messages later.

B The differences between the BSSV12 scheme and new scheme

We present the differences between the [BSSV12] scheme and our new scheme. The security of the previous scheme implies the security of the new one, from the perspective of the central authority, under the assumption that the adversary is not allowed to issue secret key queries, as stated in Definition 4. If the adversary is neither permitted to request attribute secret keys nor to contribute to the attribute secret key generation, then the result of the attribute key generation is similar to that in the previous scheme. Specifically, the values derived through the computation

$$\frac{f_{k,u}(\iota(a)) + f_{k,u}^r(\iota(a))}{t_{k,a}}$$

in the new scheme can be considered as if they were obtained from a single polynomial

$$\frac{f_{k,u}^*(\iota(a))}{t_{k,a}} := \frac{(f_{k,u} + f_{k,u}^r)(\iota(a))}{t_{k,a}}.$$

Another difference between the schemes, presented in Table 3, is about how the dummy (central authority's user specific) values are handled. Instead of publishing the dummy values $s_{k_{CA},u}P$ directly, the new scheme provides the corresponding values in two parts: first part, $\frac{s_{k_{CA},u}P}{t_{k_{CA}}}$, is shared privately with

BSSV12 scheme	New scheme
public dummy value: $s_{k_{CA},u}P$	secret value: $s_{k_{CA},u}P/t_{k_{CA}}$
public key related value P	public key related values $P, t_{k_{CA}}P$
encryption component sP	encryption components $sP, st_{k_{CA}}P$
decryption calculation $e(s_{k_{CA},u}P, sP)$	decryption calculation $e(s_{k_{CA},u}P/t_{k_{CA}}, st_{k_{CA}}P) = e(s_{k_{CA},u}P, sP)$

Table 3. Comparison table

users, while the second part, $t_{k_{CA}}P$, is shared publicly as a new attribute, where $t_{k_{CA}} \leftarrow \{1, \dots, p-1\}$. If $t_{k_{CA}} = 1$ is chosen, the new values are equal to the previous values. The additional information that becomes available to the central authority after encryption is the value $st_{k_{CA}}P$. Since the central authority has previously chosen the value $t_{k_{CA}}$, this is equivalent to knowing the value sP . To summarize, having access to the values $\frac{s_{k_{CA},u}P}{t_{k_{CA}}}$ and $t_{k_{CA}}P$, instead of the $s_{k_{CA},u}P$ values will not reveal any more information about the secret values than having access only to the $s_{k_{CA},u}P$ values.